

Sarah Benavides

CS 362 Lab 10

4/6/2026

Linux Forensics (Volatile Data Collection)

Part 1: Non-technical summary:

In this lab, we focused on learning how to collect important information from a Linux system while it is still running. This type of information, called volatile data, can disappear if the computer is turned off, so it must be captured quickly.

The lab shows us how to use simple commands to find basic details like system time, how long the system has been running, and what users are logged in. We also covered how to check network information, such as IP addresses and active connections, to better understand what the system is communicating with. This lab also touches on ways to analyze running programs, open files, and system logs. These help reveal what actions have been taken on the system and whether anything unusual is happening.

Overall, the goal of this lab is to teach us how to quickly gather and review live system data so that important information is not lost. This is useful in digital forensics when trying to understand or investigate activity on a computer.

Part 2: Technical summary:

To begin this lab, we focused on Linux file structure and important files. By inputting the command `ls -l /` we allow a list of files and directories to appear.

```
(kali@kali)-[~]
└─$ ls -l /
total 64
lrwxrwxrwx 1 root root 7 Feb 11 22:32 bin -> usr/bin
drwxr-xr-x 3 root root 4096 Feb 19 18:37 boot
drwxr-xr-x 18 root root 3380 Mar 27 13:48 dev
drwxr-xr-x 181 root root 12288 Mar 26 16:16 etc
drwxr-xr-x 3 root root 4096 Feb 11 23:13 home
lrwxrwxrwx 1 root root 33 Feb 11 22:35 initrd.img -> boot/initrd.img-6.1
6.8+kali-amd64
lrwxrwxrwx 1 root root 33 Feb 11 22:35 initrd.img.old -> boot/initrd.img
-6.16.8+kali-amd64
lrwxrwxrwx 1 root root 7 Feb 11 22:32 lib -> usr/lib
lrwxrwxrwx 1 root root 9 Feb 11 22:55 lib32 -> usr/lib32
lrwxrwxrwx 1 root root 9 Feb 11 22:32 lib64 -> usr/lib64
drwx----- 2 root root 16384 Feb 11 22:32 lost+found
drwxr-xr-x 4 root root 4096 Mar 26 16:01 media
drwxr-xr-x 2 root root 4096 Feb 11 22:32 mnt
drwxr-xr-x 3 root root 4096 Feb 11 22:54 opt
dr-xr-xr-x 234 root root 0 Mar 27 13:46 proc
drwx----- 8 root root 4096 Mar 27 13:48 root
drwxr-xr-x 38 root root 900 Mar 27 13:55 run
lrwxrwxrwx 1 root root 8 Feb 11 22:32 sbin -> usr/sbin
drwxr-xr-x 3 root root 4096 Feb 11 22:59 srv
dr-xr-xr-x 13 root root 0 Mar 27 13:46 sys
drwxrwxrwt 12 root root 320 Mar 27 13:56 tmp
drwxr-xr-x 15 root root 4096 Feb 11 22:55 usr
drwxr-xr-x 12 root root 4096 Feb 11 23:16 var
```

The next few commands include the `cat` command and allows us to display the contents of a particular file.

```

(kali㉿kali)-[~]
$ cat /proc/uptime
1251.48 2098.79

(kali㉿kali)-[~]
$ cat /proc/swaps

```

Filename	Type	Size	Used	Priority

The command **cat**

/proc/uptime shows how long the system has been running. The second command, **cat /proc/swaps** displays information about the virtual memory. The swap disk space is used when RAM is full.

```

(kali㉿kali)-[~]
$ cat /etc/resolv.conf
# Generated by NetworkManager
search cable.rcn.com
nameserver 192.168.0.1

(kali㉿kali)-[~]
$ cat /proc/version
Linux version 6.16.8+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-14 (Debian 14.3.0-8) 14.3.0, GNU ld (GNU Binutils for Debian) 2.45) #1 SMP PREEMPT_DYNAMIC Kali 6.16.8-1kali1 (2025-09-24)

```

The third cat command

we used was **cat /etc/resolv.conf** and displays the DNS nameserver and network lookup settings. The last cat command we used in part one was **cat /proc/version** and this showed the Linux kernel version being used.

In the second part of this lab, we concentrated on collecting basic volatile information. In order to collect the hostname and time information we use the following commands: **hostname**, **cat /proc/resolv.conf**, **cat/etc/timezone**, **cat /proc/uptime**, and **uptime**.

```

(kali㉿kali)-[~]
$ hostname
kali

(kali㉿kali)-[~]
$ date
Fri Mar 27 02:58:18 PM CDT 2026

(kali㉿kali)-[~]
$ cat /etc/timezone
cat: /etc/timezone: No such file or directory

(kali㉿kali)-[~]
$ cat /proc/uptime
4366.85 7076.02

(kali㉿kali)-[~]
$ uptime
14:58:57 up 1:12, 1 user, load average: 0.52, 0.55, 0.51

```

When using these commands,

I had issues with **cat /proc/resolv.conf** and **cat /etc/timezone** displaying the correct outputs. I changed **cat /proc/resolv.conf** to **date** and received the correct output, but with the **timezone** command it did not work and showed there was no such file or directory.

We then shifted slightly and began collecting basic network information. The first command we used was **ip addr**. This command allowed us to show the IP addresses of all interfaces and related information.

```

(kali@kali)-[~]
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:35:16:ea brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:db:e0:fa brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute eth1
        valid_lft 574sec preferred_lft 574sec
    inet6 fe80::a00:27ff:fedb:e0fa/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

In order to display the information for all interfaces we used the command **ip link show**.

```

(kali@kali)-[~]
$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 08:00:27:35:16:ea brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 08:00:27:db:e0:fa brd ff:ff:ff:ff:ff:ff

```

When an investigator needs to be more specific, they can use the command **ip link show dev eth1** to show information for a specific interface (eth1).

```

(kali@kali)-[~]
$ ip link show dev eth1
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 08:00:27:db:e0:fa brd ff:ff:ff:ff:ff:ff

```

If the user needs to display the interface statistics of eth1 they would input **ip -s link show dev eth1**.

```

(kali@kali)-[~]
$ ip -s link show dev eth1
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 08:00:27:db:e0:fa brd ff:ff:ff:ff:ff:ff
    RX:  bytes packets errors dropped missed mcast
         15450      63      0      0      0      0
    TX:  bytes packets errors dropped carrier collsns
         14790      96      0      0      0      0

```

In order to show the routing and ARP table we can use the command **ip route** and **ip neigh** respectfully.

```
(kali㉿kali)-[~]
$ ip route
192.168.56.0/24 dev eth1 proto kernel scope link src 192.168.56.101 metric 10
0

(kali㉿kali)-[~]
$ ip neigh
192.168.56.100 dev eth1 lladdr 08:00:27:c6:cf:47 REACHABLE
fe80::2 dev eth0 lladdr 52:56:00:00:00:02 router STALE
```

The last command used for collecting basic network information is **ss -a | head**. This command displays all socket statistics.

```
(kali㉿kali)-[~]
$ ss -a | head
RTNETLINK answers: Invalid argument
Netid State Recv-Q Send-Q Local Address Peer Address:Port
s:Port
nl UNCONN 0 0 rtn
l:NetworkManager/841 *
nl UNCONN 0 0 rtn
l:kernel *
nl UNCONN 0 0 rtn
l:xdg-desktop-por/2121 *
nl UNCONN 0 0 rtn
l:xdg-desktop-por/2121 *
nl UNCONN 0 0 rtn
l:NetworkManager/841 *
nl UNCONN 4352 0 tcpdia
g:ss/42629 *
nl UNCONN 960 0 tcpdia
g:kernel *
nl UNCONN 0 0 selinu
x:kernel *
nl UNCONN 0 0 audi
t:systemd/1 *
```

This output lists the netID, state, recv-Q, send-Q, local address, peer address, and the port process of the current network.

When an interface is in promiscuous mode, it accepts all received packets. This unfortunately can be a malicious packet sniffing technique. Using the **ifconfig** command, we can check the mode of operation of an interface and change it if need be.

```
(kali㉿kali)-[~]
$ ifconfig lo
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

This output contains a list of information regarding the loopback network interface within the system being used. This list includes the IP address, network range, data received and sent, and more.

In order to set the interface **lo** to promiscuous mode, we can use the following command: **sudo ifconfig lo promisc**. After successfully changing the interface to promiscuous mode, we input the **ifconfig lo** command once more to experiment with it in this specific mode.

```

(kali@kali)-[~]
$ sudo ifconfig lo promisc
[sudo] password for kali:

(kali@kali)-[~]
$ ifconfig lo
lo: flags=329<UP,LOOPBACK,RUNNING,PROMISC> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

The output of the command

ifconfig lo in promiscuous mode was generally the same as using the command without it. The only difference was that the number of flags increased when in promiscuous mode. After the user is done entering commands within promiscuous mode, we used the following command to disable that mode on the **lo** interface: **sudo ifconfig lo -promisc**.

```

(kali@kali)-[~]
$ sudo ifconfig lo -promisc

```

We were able to verify that the promiscuous mode was disabled by inputting the same command we used to list the **lo** interface information.

```

(kali@kali)-[~]
$ ifconfig lo
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

The next command we tested was the **dmesg** command. **Dmesg** is used to retrieve messages from the kernel to help investigators track actions performed on the machine they are investigating. This command lists all messages from the start of the kernel. We used the following input to display the first five lines of the output: **sudo dmesg | head -n 5**.

```

(kali@kali)-[~]
$ sudo dmesg | head -n 5
[ 0.000000] Linux version 6.16.8+kali-amd64 (devel@kali.org) (x86_64-linux
-gnu-gcc-14 (Debian 14.3.0-8) 14.3.0, GNU ld (GNU Binutils for Debian) 2.45)
#1 SMP PREEMPT_DYNAMIC Kali 6.16.8-1kali1 (2025-09-24)
[ 0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-6.16.8+kali-amd64 root=
UUID=a4927ada-29f9-4818-95df-476de1815d27 ro quiet splash
[ 0.000000] BIOS-provided physical RAM map:
[ 0.000000] BIOS-e820: [mem 0x0000000000000000-0x000000000009fbff] usable
[ 0.000000] BIOS-e820: [mem 0x000000000009fc00-0x000000000009ffff] reserve
d

```

In order to display the first two lines of the **dmesg** output we can use the command **sudo dmesg -T | head -n 2**. We were able to show the output in a human readable timestamp format by using the option **-T**.

```

(kali@kali)-[~]
$ sudo dmesg -T | head -n 2
[Sun Mar 29 15:16:44 2026] Linux version 6.16.8+kali-amd64 (devel@kali.org) (
x86_64-linux-gnu-gcc-14 (Debian 14.3.0-8) 14.3.0, GNU ld (GNU Binutils for De
bian) 2.45) #1 SMP PREEMPT_DYNAMIC Kali 6.16.8-1kali1 (2025-09-24)
[Sun Mar 29 15:16:44 2026] Command line: BOOT_IMAGE=/boot/vmlinuz-6.16.8+kali
-amd64 root=UUID=a4927ada-29f9-4818-95df-476de1815d27 ro quiet splash

```

The last option of the **dmesg** command we visited was **sudo dmesg -T | grep "device lo"**. This command displays lines of output that contain the word "lo:".

```

(kali@kali)-[~]
$ sudo dmesg -T | grep "lo:"
[Sun Mar 29 15:54:41 2026] lo: entered promiscuous mode
[Sun Mar 29 16:01:34 2026] lo: left promiscuous mode
[Sun Mar 29 16:19:44 2026] lo: entered promiscuous mode
[Sun Mar 29 16:19:49 2026] lo: left promiscuous mode

```

Within the same part of this lab, we shifted to examining the command **lsuf**. This command is used to list open files within the device being used. Using the command **sudo lsuf | head -n 4**, we were able to list the first four open files.

```

(kali@kali)-[~]
$ sudo lsuf | head -n 4
lsuf: WARNING: can't stat() fuse.gvfsd-fuse file system /run/user/1000/gvfs
Output information may be incomplete.
lsuf: WARNING: can't stat() fuse.portal file system /run/user/1000/doc
Output information may be incomplete.
COMMAND  PID  TID TASKCMD  USER  FD  TYPE  DEVICE
SIZE/OFF  NODE NAME
systemd  1   2 /          root  cwd  DIR   8,1
4096
systemd  1   2 /          root  rtd  DIR   8,1
4096
systemd  1   2 /          root  txt  REG   8,1
141696 1182043 /usr/lib/systemd/systemd

```

This output displays the file's command, PID, TID, user, type, device, and more. When we input the command **sudo lsuf -i**, we are looking to reveal the files opened by network connections. If we wanted to search for the files opened by a given user, we would use the command **sudo lsuf -u kali | wc -l**.

```

(kali@kali)-[~]
$ sudo lsuf -i
COMMAND  PID  USER  FD  TYPE  DEVICE  SIZE/OFF  NODE NAME
NetworkMa 773 root 26u IPv4  8164      0t0  UDP 192.168.56.101:bootpc→192.
168.56.100:bootps

(kali@kali)-[~]
$ sudo lsuf -u kali | wc -l
lsuf: WARNING: can't stat() fuse.gvfsd-fuse file system /run/user/1000/gvfs
Output information may be incomplete.
lsuf: WARNING: can't stat() fuse.portal file system /run/user/1000/doc
Output information may be incomplete.
5027

```

While using the same **lsuf** command, we next want to display the first five lines from the returned results. We can do so by inputting **sudo lsuf -u kali | head -n 5**.

```
(kali@kali)-[~]
└─$ sudo lsof -u kali | head -n 5
lsof: WARNING: can't stat() fuse.gvfsd-fuse file system /run/user/1000/gvfs
Output information may be incomplete.
lsof: WARNING: can't stat() fuse.portal file system /run/user/1000/doc
Output information may be incomplete.
COMMAND      PID USER  FD      TYPE          DEVICE  SIZE/OFF      NODE NAME
ME
systemd      1051 kali cwd          DIR           8,1      4096          2 /
systemd      1051 kali rtd          DIR           8,1      4096          2 /
systemd      1051 kali txt          REG           8,1     141696     1182043 /u
sr/lib/systemd/systemd
systemd      1051 kali mem          REG           8,1     182560     1179670 /u
sr/lib/x86_64-linux-gnu/libseccomp.so.2.6.0
```

In order to display all the opened files by a particular process, we would use the command **sudo lsof -c ssh**.

```
└─$ sudo lsof -c ssh
lsof: WARNING: can't stat() fuse.gvfsd-fuse file system /run/user/1000/gvfs
Output information may be incomplete.
lsof: WARNING: can't stat() fuse.portal file system /run/user/1000/doc
Output information may be incomplete.
COMMAND      PID USER  FD      TYPE          DEVICE  SIZE/OFF      NODE NAME
ssh-agent    1242 kali cwd          DIR           8,1      4096          2 /
ssh-agent    1242 kali rtd          DIR           8,1      4096          2 /
ssh-agent    1242 kali txt          REG           8,1     424320     1204362 /usr/bin/s
sh-agent
ssh-agent    1242 kali mem          REG           8,1     141224     1177697 /usr/lib/x
86_64-linux-gnu/oss1-modules/legacy.so
ssh-agent    1242 kali mem          REG           8,1     2003408     1180292 /usr/lib/x
86_64-linux-gnu/libc.so.6
ssh-agent    1242 kali mem          REG           8,1     6374024     1179704 /usr/lib/x
86_64-linux-gnu/libcrypto.so.3
ssh-agent    1242 kali mem          REG           8,1     821240     1177703 /usr/lib/x
86_64-linux-gnu/libzstd.so.1.5.7
ssh-agent    1242 kali mem          REG           8,1     125376     1214775 /usr/lib/x
86_64-linux-gnu/libz.so.1.3.1
ssh-agent    1242 kali mem          REG           8,1     225600     1180287 /usr/lib/x
86_64-linux-gnu/ld-linux-x86-64.so.2
ssh-agent    1242 kali 0u        CHR           1,3         0t0          4 /dev/null
ssh-agent    1242 kali 1u        CHR           1,3         0t0          4 /dev/null
ssh-agent    1242 kali 2u        CHR           1,3         0t0          4 /dev/null
ssh-agent    1242 kali 3u        unix 0x00000000bb140366 0t0     11401 /home/kali
/.ssh/agent/s.RDjpj98jBr.agent.0B8CJP673h type=STREAM (LISTEN)
```

We then experimented with a few different commands to collect different forms of volatile information. The first being **mount**. This command lists the mounted file systems and the corresponding mounting directories. We input **mount -l** to generate this list.

```
(kali@kali)-[~]
└─$ mount -l
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
udev on /dev type devtmpfs (rw,nosuid,relatime,size=1900468k,nr_inodes=475117
,mode=755,inode64)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=600,ptmx
mode=000)
tmpfs on /run type tmpfs (rw,nosuid,nodev,noexec,relatime,size=401344k,mode=7
55,inode64)
/dev/sda1 on / type ext4 (rw,relatime,errors=remount-ro)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,re
latime)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64)
cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime,nsdel
egate,memory_recursiveprot,memory_hugetlb_accounting)
none on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime)
bpf on /sys/fs/bpf type bpf (rw,nosuid,nodev,noexec,relatime,mode=700)
systemd-1 on /proc/sys/fs/binfmt_misc type autofs (rw,relatime,fd=41,prgrp=1,t
imeout=0,minproto=5,maxproto=5,direct,pipe_ino=5488)
hugetlbfs on /dev/hugepages type hugetlbfs (rw,nosuid,nodev,relatime,pagesize
=2M)
debugfs on /sys/kernel/debug type debugfs (rw,nosuid,nodev,noexec,relatime)
mqueue on /dev/mqueue type mqueue (rw,nosuid,nodev,noexec,relatime)
tracefs on /sys/kernel/tracing type tracefs (rw,nosuid,nodev,noexec,relatime)
fusectl on /sys/fs/fuse/connections type fusectl (rw,nosuid,nodev,noexec,rela
time)
none on /run/credentials/systemd-journald.service type tmpfs (ro,nosuid,nodev
```

This output was a bit lengthy, so I only included the first part of it in the screenshot above.

The next eight commands we examined are as follows: **df**, **lsmod**, **modinfo**, **insmod**, **rmmod**, **ps**, **pmap**, and **strace**. The command **df** displays the amounts of free and used space on the mounted file systems, and the mounting directories.

```

(kali@kali)-[~]
$ df
Filesystem      1K-blocks      Used Available Use% Mounted on
udev            1900468          0   1900468   0% /dev
tmpfs           401344          980    400364   1% /run
/dev/sda1       24253528  19402096   3594064  85% /
tmpfs           2006700          4    2006696   1% /dev/shm
none            1024           0      1024   0% /run/credentials/systemd-journald.service
tmpfs           2006704        324    2006380   1% /tmp
work            248651772  243096852  5554920  98% /media/sf_work
none            1024           0      1024   0% /run/credentials/getty@tty1.service
tmpfs           401340         100     401240   1% /run/user/1000

```

This output includes the files system name, 1K-blocks, how much storage is used and available, and where the files are mounted on. The second command we looked at is **lsmod**. This command displays the kernel loaded modules. By using the command **lsmod | head -n 6** we are able to list the first six lines of the output.

```

(kali@kali)-[~]
$ lsmod | head -n 6
Module                Size  Used by
snd_seq_dummy         12288  0
snd_hrtimer           12288  1
snd_seq              118784  7 snd_seq_dummy
snd_seq_device        16384  1 snd_seq
rfkill                45056  2

```

This output displays the module name, size and what it is used by. The third command used is **modinfo**. This presents the information of a particular module. The **vsock** module enables the communication between the host machine and the virtual machine.

```

(kali@kali)-[~]
$ modinfo vsock
filename:             /lib/modules/6.16.8+kali-amd64/kernel/net/vmw_vsock/vsock.ko.xz
license:              GPL v2
version:              1.0.2.0-k
description:          VMware Virtual Socket Family
author:               VMware, Inc.
srcversion:           8ABA83FF2FD1E92D49ADC5F
depends:
intree:               Y
name:                 vsock
retpoline:            Y
vermagic:             6.16.8+kali-amd64 SMP preempt mod_unload modversions
sig_id:               PKCS#7
signer:               Build time autogenerated kernel key
sig_key:              6A:9A:F7:D9:0F:2B:6B:F6:55:D5:D1:67:D5:77:B7:90:01:C6:DC:BF
sig_hashalgo:         sha256
signature:            30:65:02:31:00:E9:92:EC:47:F4:9D:2D:C8:AE:82:61:06:C2:83:61:FC:B9:4B:75:36:9A:9B:E3:A9:5A:B5:06:8E:5F:1E:FA:65:21:32:3E:75:F2:22:5E:71:C4:D9:3B:1E:19:C4:12:96:02:30:23:04:76:20:1E:8E:91:41:03:A1:B7:D8:6F:48:3B:68:CD:31:EA:7F:98:4C:5D:BD:C2:E7:46:FC:48:CF:17:1C:9E:C9:3D:DC:09:DB:1D:08:46:D9:21:DF:E7:2A:44:F5

```

The output of this command lists the filename, version number, a brief description, the author, and more.

The following two commands, **insmod** and **rmmod**, are used to insert modules and remove modules from the kernel, respectively. The sixth command we looked at is **ps**. **Ps** is

used to obtain various details on processes. In order to do so, we will use a few different options. To begin, we input **ps** by itself. This shows the processes for the current terminal.

```
(kali㉿kali)-[~]
$ ps
  PID TTY          TIME CMD
 8100 pts/0        00:00:05 zsh
54194 pts/0        00:00:00 ps
```

The first option we used was **ps -T**. This displays the processes for the current terminal and lists the PID, server PID, the terminal type the user is logged into, the time and command.

```
(kali㉿kali)-[~]
$ ps -T
  PID   SPID TTY          TIME CMD
 8100   8100 pts/0        00:00:05 zsh
55395  55395 pts/0        00:00:00 ps
```

The second option we looked at was **ps -e | head**. This command lists all running processes on the current device.

```
(kali㉿kali)-[~]
$ ps -e | head
  PID TTY          TIME CMD
    1 ?            00:00:18 systemd
    2 ?            00:00:00 kthreadd
    3 ?            00:00:00 pool_workqueue_release
    4 ?            00:00:00 kworker/R-rcu_gp
    5 ?            00:00:00 kworker/R-sync_wq
    6 ?            00:00:00 kworker/R-kvfree_rcu_reclaim
    7 ?            00:00:00 kworker/R-slub_flushwq
    8 ?            00:00:00 kworker/R-netns
   12 ?            00:00:00 kworker/u8:0-ipv6_addrconf
```

The third option of **ps** we examined was **ps -x | head -n 5**. This command lists all processes associated with the current user.

```
(kali㉿kali)-[~]
$ ps -x | head -n 5
  PID TTY      STAT   TIME COMMAND
 1051 ?        Ss      0:01  /usr/lib/systemd/systemd --user
 1053 ?        S        0:00  (sd-pam)
 1075 ?        Ss      0:02  /usr/bin/dbus-daemon --session --address=systemd
: --nofork --nopidfile --systemd-activation --syslog-only
 1076 ?        Ssl     0:00  /usr/bin/pipewire
```

The next option we used was **ps -p 1644 1669 8093**. This lists processes by process ID.

```
(kali㉿kali)-[~]
$ ps -p 1644 1669 8093
  PID TTY      STAT   TIME COMMAND
 1644 ?        Ssl     0:01  /usr/libexec/gvfs-gphoto2-volume-monitor
 1669 ?        Ssl     0:01  /usr/libexec/gvfs-afc-volume-monitor
 8093 ?        Sl      0:19  /usr/bin/qterminal
```

The fourth option used in this part was **ps -A -o user,pid,ppid,sess,args | head**. This command lists the processes by username, pid, ppid, session id, and arguments.

```

(kali@kali)-[~]
$ ps -A -o user,pid,ppid,sess,args | head
USER      PID    PPID   SESS  COMMAND
root        1        0        1  /sbin/init splash
root        2        0        0  [kthreadd]
root        3        2        0  [pool_workqueue_release]
root        4        2        0  [kworker/R-rcu_gp]
root        5        2        0  [kworker/R-sync_wq]
root        6        2        0  [kworker/R-kvfree_rcu_reclaim]
root        7        2        0  [kworker/R-slub_flushwq]
root        8        2        0  [kworker/R-netns]
root       12        2        0  [kworker/u8:0-ipv6_addrconf]

```

The last variation of **ps** we looked at was **ps -s 1094 | head**. This command displays a list of processes attributed to a particular session.

```

(kali@kali)-[~]
$ ps -s 1094 | head
  PID TTY          TIME CMD
 1094 ?           00:00:09 xfce4-session
 1254 ?           00:01:12 xfwm4
 1279 ?           00:00:04 xfsettingsd
 1286 ?           00:00:39 xfce4-panel
 1295 ?           00:00:05 Thunar
 1305 ?           00:00:09 xfdesktop
 1306 ?           00:00:02 wrapper-2.0
 1312 ?           00:02:50 wrapper-2.0
 1316 ?           00:00:05 wrapper-2.0

```

The seventh command we examined was **pmap**. **Pmap** is used to report on the memory map of a particular process. We used the command **pmap -p 1316 | head** to display the memory layout of the process with the PID 1316.

```

(kali@kali)-[~]
$ pmap -p 1316 | head
1316:  /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-lin
ux-gnu/xfce4/panel/plugins/libsystray.so 14 25165837 systray Status Tray Plug
in Provides status notifier items (application indicators) and legacy systray
items
00005602a373f000    28K r----- /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-
2.0
00005602a3746000    24K r-x--- /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-
2.0
00005602a374c000    16K r----- /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-
2.0
00005602a3750000     4K r----- /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-
2.0
00005602a3751000     4K rw---- /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-
2.0
00005602af47f000   4792K rw---- [ anon ]
00007fc3d0000000    132K rw---- [ anon ]
00007fc3d0021000   65404K ----- [ anon ]
00007fc3d4000000    132K rw---- [ anon ]

```

The last command we focused on in this part of the lab was **strace**. **Strace** is used to trace the system calls and signals issued by a specific process. Using the command **ps -A | grep zsh** allows us to find the PID of the Z shell and trace its system call from another terminal.

```

(kali@kali)-[~]
$ ps -A | grep zsh
1832 pts/0    00:00:00 zsh
1833 pts/1    00:00:00 zsh

```

We then took the information from that output and used it to trace a system call from the second terminal using **strace -p 1832**.

```

(kali@kali)-[~]
$ strace -p 1832
strace: Process 1832 attached
read(10, "l", 1) = 1
rt_sigprocmask(SIG_BLOCK, [WINCH], [], 8) = 0
fcntl(0, F_DUPFD, 10) = 11
close(0) = 0
openat(AT_FDCWD, "/dev/null", O_RDWR|O_NOCTTY) = 0
mmap(NULL, 16384, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0
x7f143c0f4000
rt_sigprocmask(SIG_BLOCK, [CHLD], [WINCH], 8) = 0
mmap(NULL, 16384, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0
x7f143c0eb000
rt_sigprocmask(SIG_BLOCK, [CHLD], [CHLD WINCH], 8) = 0
rt_sigprocmask(SIG_BLOCK, [CHLD], [CHLD WINCH], 8) = 0
rt_sigprocmask(SIG_UNBLOCK, [CHLD], [CHLD WINCH], 8) = 0
rt_sigprocmask(SIG_BLOCK, [CHLD], [WINCH], 8) = 0
mmap(NULL, 16384, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0
x7f143bd0a000
rt_sigprocmask(SIG_BLOCK, [CHLD], [CHLD WINCH], 8) = 0
rt_sigprocmask(SIG_BLOCK, [CHLD], [CHLD WINCH], 8) = 0
rt_sigprocmask(SIG_UNBLOCK, [CHLD], [CHLD WINCH], 8) = 0
rt_sigprocmask(SIG_BLOCK, [CHLD], [WINCH], 8) = 0
rt_sigprocmask(SIG_BLOCK, [CHLD], [CHLD WINCH], 8) = 0
rt_sigprocmask(SIG_UNBLOCK, [CHLD], [CHLD WINCH], 8) = 0
munmap(0x7f143bd0a000, 16384) = 0
rt_sigprocmask(SIG_BLOCK, [CHLD], [WINCH], 8) = 0
mmap(NULL, 16384, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0

```

After inputting the previous command, we went back to the first terminal and input the **ls** command. Once we entered **ls**, the screenshot above appeared in the second terminal. The output was rather long, so I only included the beginning of the output.

The third part of this lab focused on Linux firewalls, SSH service, and port scanning using **nmap**. I created two separate Linux virtual machines: one with **UFW** and **SSH** services, and the other with **Parrot Security**. After that step, I accessed the Kali Linux virtual machine and input the following command: **nmap -sT scanme.nmap.org**. This command is used for port scanning, specifically the TCP ports.

```

(kali@kali)-[~]
$ nmap -sT scanme.nmap.org
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-30 18:45 -0500
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.064s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
9929/tcp  open  nping-echo
31337/tcp open  Elite
Nmap done: 1 IP address (1 host up) scanned in 9.67 seconds

```

This output lists the specific ports, state, and services. The next tool we used in this part was the **UFW firewall**. Before this step, I already downloaded the packages, so I moved on to the next command. I input the command **sudo systemctl status ufw**. This command displays the status of the **UFW** service using the **systemctl** command.

```

(kali@kali)-[~]
$ sudo systemctl status ufw
[sudo] password for kali:
o ufw.service - Uncomplicated firewall
   Loaded: loaded (/usr/lib/systemd/system/ufw.service; enabled; preset: enabled)
   Active: inactive (dead)
     Docs: man:ufw(8)

```

We then used the command **sudo systemctl start ufw** to begin using the **UFW** service since it was installed but just not being used in the previous command.

```

(kali@kali)-[~]
$ sudo systemctl start ufw

(kali@kali)-[~]
$ sudo systemctl status ufw
● ufw.service - Uncomplicated firewall
   Loaded: loaded (/usr/lib/systemd/system/ufw.service; enabled; preset: ena
   Active: active (exited) since Mon 2026-03-30 18:58:31 CDT; 26s ago
   Invocation: a9a1a77d475345e2858307118911ca49
     Docs: man:ufw(8)
   Process: 57521 ExecStart=/usr/lib/ufw/ufw-init start quiet (code=exited,
 Main PID: 57521 (code=exited, status=0/SUCCESS)
    Mem peak: 1.8M
       CPU: 82ms

Mar 30 18:58:31 kali systemd[1]: Starting ufw.service - Uncomplicated firewa
Mar 30 18:58:31 kali systemd[1]: Finished ufw.service - Uncomplicated firewa

```

After starting the UFW service, we input the same status command we used before to display an active status versus an inactive status. In order to display the status of the firewall, we used the command **sudo ufw status**.

```

(kali@kali)-[~]
$ sudo ufw status
Status: active

```

This output stated the status of the firewall was active. To list more details of the firewall status, we input the command **sudo ufw status verbose**.

```

(kali@kali)-[~]
$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

```

The output of this command showed that the firewall is on, logs/records basic activities, and denies incoming connections but still allows outgoing connections.

The next command we focused on was **sudo ufw allow 22/tcp**. This command allows us to add the TCP protocol on port 22.

```

(kali@kali)-[~]
$ sudo ufw allow 22/tcp
[sudo] password for kali:
Rule added
Rule added (v6)

```

After adding that rule, we want to look back at the details of the firewall once more. To do this, we used the same command **sudo ufw status verbose**.

```

(kali@kali)-[~]
$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)

```

This output is different from the one we originally obtained. Here, the connection allows any device to connect to this machine on TCP port 22 specifically. To take it one step further, we use the command **sudo nmap -sT localhost** to list the open ports on the local host.

```

(kali@kali)-[~]
$ sudo nmap -sT localhost
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-30 20:29 -0500
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00013s latency).
Other addresses for localhost (not scanned): ::1
All 1000 scanned ports on localhost (127.0.0.1) are in ignored states.
Not shown: 1000 closed tcp ports (conn-refused)

Nmap done: 1 IP address (1 host up) scanned in 0.15 seconds

```

This output shows that all scanned ports on the localhost are being ignored because the firewall is either filtering or blocking the incoming connections.

We then wanted to check the status of the ports set by the firewall on the first virtual machine by inputting the command **sudo nmap -sT 192.168.0.148** into the second virtual machine that has Parrot Security. The IP address at the end of the command belongs to the first virtual machine.

```

[live@parrot]-[~]
$ sudo nmap -sT 192.168.0.148
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-01 00:45 UTC
Nmap scan report for kali.cable.rcn.com (192.168.0.148)
Host is up (0.00067s latency).
All 1000 scanned ports on kali.cable.rcn.com (192.168.0.148) are in
ignored states.
Not shown: 1000 filtered tcp ports (no-response)

```

The output for this command displays a similar output to that of the previous command. The output should list port 22 closed, but it only states that the 1000 filtered TCP ports are not shown.

The last tool we use in this lab was the **Secure Shell (SSH) protocol**. Network administrators use this protocol to manage remote systems. Since I already installed the open-source shell tool, I skipped right to inputting the command **sudo systemctl status ssh | grep -I active**. This command displays the status of the ssh service.

```

(kali@kali)-[~]
$ sudo systemctl status ssh | grep -i active
[sudo] password for kali:
Active: inactive (dead)

```

Since the output states that the service is inactive, we need to turn it on using the command **sudo systemctl start ssh**.

```

(kali@kali)-[~]
$ sudo systemctl start ssh

(kali@kali)-[~]
$ sudo systemctl status ssh | grep -i active
Active: active (running) since Tue 2026-03-31 17:01:59 CDT; 10s ago

```

After successfully starting the **ssh service**, we checked the status of the **ssh service** again, and this time it was active. While using the same IP address as before, we entered the command **sudo nmap -sT 192.168.0.148** into the second virtual machine to check the status of the ports set by the firewall on the first virtual machine again.

```

[live@parrot]~$ sudo nmap -sT 192.168.0.148
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-01 00:52 UTC
Nmap scan report for kali.cable.rcn.com (192.168.0.148)
Host is up (0.013s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh

Nmap done: 1 IP address (1 host up) scanned in 5.13 seconds

```

This showed that port 22 is the only open port open for connection.

The last tool we used in this lab was the **openssh tool**, which, when used correctly, allows us to establish a secure connection between the two virtual machines. I input the following command in the second virtual machine to perform this task: **ssh kali@192.168.0.148**.

```

[live@parrot]~$ ssh kali@192.168.0.148
The authenticity of host '192.168.0.148 (192.168.0.148)' can't be established.
ED25519 key fingerprint is SHA256:rqMJsfrCaQfjV7DSVeknBz55knPJxxiICD3st+p3SpE.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.0.148' (ED25519) to the list of known hosts.
kali@192.168.0.148's password:
Linux kali 6.16.8+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.16.8-1kali1 (2025-09-24) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.

```

The output I received shows that the connection was successfully established. In order to create the connection, the other way around, we need to input the following command into the second virtual machine: **echo Test from the Parrot Security host > testParrotKali.txt**.

```

(kali@kali)~$ echo Test from the Parrot Security host > testParrotKali.txt

```

To confirm this connection, we input **ls** on the first virtual machine and analyze the output.

```

(kali@kali)~$ ls
Desktop      linux_forensics.000  Music        testdoc.txt
Documents    linux_forensics.info Pictures      testParrotKali.txt
Downloads    linux_forensics.log  Public       testtest.txt
evidencesdb1 linux_image.dd       sample.txt   test.txt
file.txt     mbr.image            Templates    Videos

```

In this list of files, the file **testParrotKali.txt** appears successfully. We intended this to happen to prove that we were able to manipulate files on one computer while using another.

Part 3: References

1. Lab 10: Linux Forensics (Volatile Data Collection)
2. [Cat command variations](#)
3. [Ifconfig command variations](#)